
# Computer Games Development
# Project Report
# Year IV

[Jake Fitzgerald]
[C00288105]
[13/042026]

[Declaration form to be attached]

**Contents**

**Acknowledgements**

Thank you to my supervisor Amr Abdelhafez for the constant feedback and support offered during the development of this project. I also thank the Carlow SETU for providing the resources available to assist me during the project.

**Project Abstract**

This project is to fully explore the Midi format in many aspects (writing, reading, recording and listening) to develop an interactive app similar to the game Guitar Hero but for piano. Using the Midi format we can create an app to help users learn to play the piano using their Midi keyboard device.

The user can learn from preset songs or load in their own songs, including songs they can record themselves. They receive statistical feedback on their performance such as their timings of the key presses and  hit notes accuracy.

Using different assists can aid the user such as a Practice Mode to allow for them to focus on which keys to press instead of timing and an Easy Input Mode to allow for more generous timing when pressing a key.

The user can upload a recorded session to a server's leaderboard which can be viewed during the game and download those user's performance that are stored in the midi format. This can be used in the Ghost Mode where they see this user's or their own performance during a song as an overlay of how accurate their performance was.

With all of these combined is a piano learning application that incorporates gameplay, basic server pushing and fetching, reading and writing to Midi and real time Midi input handling.

**Project Introduction**

This project's goal is to understand and fully utilise the Midi (Musical Instrument Digital Interface) format for the creation of an application that can teach a user to play the piano. To do this we will need to create several classes to be able to understand the Midi format (read, write, and input listening). After we have gathered our Midi data we can use it to generate gameplay entities such as notes that spawn and move downwards in timing with the song's BPM (Beats Per Minute). We can then use this along with a Midi input handler to listen to which keys are being pressed to work in tandem with the gameplay.

While there are existing libraries for a variety of purposes in relation to the Midi format, for this project we will be creating each of these elements from scratch. Doing it this way allows for this project to fully understand how to build these different Midi tools from the ground up

The end result is a free version of a game such as Guitar Hero or other piano learning applications such as Synthesia.

Main contributions in the project:

**- Midi Parser:** Read the Midi format to extract the different track data that can be used to populate the falling note entities in gameplay, control the playback timing with the BPM/ Tempo tracks.

**- Midi Writer:** Writes the different tracks from which song was being used during gameplay such as it's BPM/Temp and the recorded notes during gameplay which then populates the third track of the file. This can then be used in the Ghost Mode as a visual way to show your performance.

**- Midi Listener**: An event listener that runs on a seperate thread to detect when different keys are pressed. This tracks the note's value and velocity which is then passed to gameplay and different visualisers.

**- Server:** A Microsoft Azure server that stores the database that is used to populate the game's leaderboard data. It allows a user to upload their performances associated with their username as well as the actual recording of the performance in a Midi file that can be downloaded and loaded into the game.

## Literature Review

Researching different attempts at creating an application for utilising different Midi operations was difficult due to majority of them only focus on a single action instead of several (i.e. only reads Midi but not listening).The most influential I had found was Synthesia for it achieving the core concept of a game like Guitar Hero had done on consoles. The functionality of being able to load in any custom Midi file you want and to learn how to play it was very appealing.

However it failed to integrate other important features such as recording Midi data which was a missed opportunity for a user to be able to record their performance, with this they could then upload it to a leaderboard and allow other users to download it to see how well or poorly they performed. Synthesia also has no leaderboard making it feel like an isolated experience without any form of competition of how other users are playing and progressing.

As for learning the fundamentals of Midi archtiecture I have listed in the References section various links for what assisted me during this project, most notably “Midi Programming – A complete study by Stephane Richard”.

## Evaluation and Discussion

By adding in a way for the user to record their performance and share it with others in an online community, this would be more beneficial and help foster an encouraging and competitive community of users to assist in their journeys of learning to play the piano. Which goes beyond what Synthesia or other piano learning applications can provide.

This helps improve the goal of the application of learning how to play the piano since there are nó extrinsic factors (friends, rivals, world records, etc) that influence the motivation for learning to play a specific song (intrinsic factor).

This however may possibly have the opposite of discouraging a user, due to them feeling like they have low skill level (compared to the top scoring users worldwide) and feel like they haven’t progressed at all when they see other much more skillful user’s performances.

Ideally other gamey type elements could be added such as a leveling up system that can increase a user’s rank. This way they will always feel like they are progressing due to this system constantly rewarding them even just for attempting to learn a song with experience points which increase a user’s level. Thus making them feel like they have improved, regardless if their performance was better or worse than their previous personal best.

**Project Milestones**

Milestone 1:

- Basic form of Midi parsing to fill in the different track data to be used later in gameplay.

- Visualisers that will be used later for testing the Midi input handling and Sound Manager.

Milestone 2:

- Basic Midi input handling to be used in the visualiser's scenes. The visualisers are a basic implementation of several components that can be expanded on and used for gameplay later.

(i.e. Keyboard object class)

Milestone 3:

- Gameplay elements created and more advanced Mid parsing/ Midi input handling to be used as data passed to this scene.

Milestone 4:

- Server that holds a database that is used for storing the performances of the users that upload to it. Fetching and pushing functionality for basic id, username, score.

- Basic leaderboard that fetches the database on the server and populates the UI with the data.

Milestone 5:

- Advanced gameplay implementation for reading and generating the gameplay entities from the Midi parser and then storing the recorded notes in a vector that is then used by the Midi writer.
- Midi writing to write the recorded performance to be used as local storage or uploading it to the server's database as a blob.

Milestone 6:

- Implementing different assistant modes such as Practice Mode, Easy Input Mode and Ghost Mode for reviewing a user's performance of a song.

- Advanced leaderboard for storing the different statistical data and have it assigned based on individual users that are associated with a song, instead of them as isolates as before.

- General bug fixing and application layout for an intuitive user experience.

**Major Technical Achievements**

Creating a way to handle Midi through parsing, writing, and listening for Midi inputs.

Gameplay that accurately shows the parsed Midi data. Midi data that can be recording during gameplay which can be then saved locally or uploaded to the server.

Storing the saved scores and Midi data on a server's database that can be fetched and download the Midi files to then be used in gameplay.

**Project Review**

*What went right?*

I feel I accomplished what the design of the application as a concept for being able to use Midi to learn to play different song on the piano.

*What went wrong?*

Reading the Midi data as a stream for both parsing and writing was very difficult due to how often it would corrupt the data that parsed or the midi that was written to. While I seperated the class in the distinct functions to avoid corruption it still occurred and took a lot of time to debug and find out why.

A major issue was when writing to the Midi file I had to create a buffer to calculate how large the third track was (note track). However I couldn't write this immediately since every performance's recorded note vector is different. So I had to manually call when to write which byte ( Note On/Note Off, Control Change, Program Change (before and after note palcement) ) and then add that size amount to the buffer's size.

The VLQ for the note's deltatime for when it plays and stops and the ordering for which one comes after or before was quite difficult due to compression and how they should be ordered by their timestamps. Originally I made a test vector of notes and while this worked for the test, the actual recorded notes didn't due to me not realising that the note's sequence is dependent on their event times and not the actual sequence of notes that were recorded in the vector. This is becuase you can hold a note and play other notes during the same duration before Note Off is ever called. So I should have been looking at the timestamps only to fix this bug sooner.

Another issue was the Ghost recorded data not spawning in the correct position during gameplay due to me only reading their timestamps and using the normal Falling note's speed and song's BPM to spawn them. However they were always offset upwards on the y-axis becuase I was never taking into account the added travel time it takes for the Falling notes to be spawn above the Keyboard input trigger and not the bottom of the screen. Minusing the travel time to the Ghost note's positions fixed this issue, but it took quite while for me to realise what the issue was.

*Conclusion*

If another person was going to tackle a project dealing with Midi they should try and create an error checking system to handle the debugging process. I mostly used breakpoints, printing to the console and looking at the Midi data as raw bytes in a hex editor called "ImHex".

*What (if anything) is still outstanding/missing (i.e., still left to do)?*

I wanted to add a piano roll editor that is commonly found in DAWs (Digital Audio Workstations) and Midi editing software. This would allow a user to manually place notes on a grid, set their lengths, the BPM/Tempo and Time Signature of the song and the note's velocity for how loud the note should be.
However while I planned this on paper I ran out of time debugging and fixing issues in the project to every start coding this feature. Also the surpervisors recommending me not to pursue this feature so late into the project, and that was the correct decision.

*Were your technology choices the right or wrong ones?*

I spend quite a while manually editing the UI for the game because of SFML, whereas if I used a different visual library I could have spent less time on this if it were more geared towards artists (drag and drop, automatic resizing, slotting UI elements near one another, etc).

However SMFL's sound functionality worked perfect with my sound pool and never caused issues for crushed audio or latency when playing a sound.

**Conclusions**

I feel I have created an application that fulfils what it set out to do and added extra features such as a server for uploading and download user's performances. I didn't have a clear vision for what the project was going to be for quite a while, other than it would explore the Midi format. Ending up on a piano learning application was the best choice for this goal and I feel I have learned everything a student would need to know about the Midi format to tackle an real life assignment for a job that incorporates (DAW software creation – FL studio, Ableton, Reason, etc)

**Future Work**

The student could focus on incorporating a type of Midi editor (usually named "Piano Roll" in Digital Audio Workstations), which would aid in the visual display and testing of reading, writing and recording Midi. Doing this would allow them to not need to use external programs such "Midi Editor" or a DAW such as FL Studio where they have to load the Midi file in manually.

Doing this would allow more freedom in the type of Midi they could create and manipulate such as easily allowing a user to create a song by placing notes down on a grid, changing the BPM on the fly, Time Signatures and velocity for how loud or soft a note can be.

**References**

Official Midi Association:
https://midi.org/

Official Documentation:

https://midi.org/spec-detail

**Windows**

Windows Multimedia Library (Midi Input Handling, MCI Midi playback)

https://learn.microsoft.com/en-us/windows/win32/multimedia/midi-functions

https://learn.microsoft.com/en-us/windows/win32/multimedia/musical-instrument-digital-interface--midi

https://learn.microsoft.com/en-us/windows-hardware/drivers/audio/audio-device-messages-for-midi

Media Control Interface (Midi playback)

https://learn.microsoft.com/en-us/windows/win32/multimedia/media-control-interface--mci

https://learn.microsoft.com/en-us/windows/win32/multimedia/using-the-mci-midi-sequencer

https://learn.microsoft.com/en-us/windows/win32/multimedia/sending-a-command

**Books:**
A Beginner's Guide to Midi – R.A. Penfold
(1993 Bernard Babani LTD)

**Applications:**

Synthesia:

https://synthesiagame.com/

Openthesia (free open-source version of Synthesia)
https://openthesia.pages.dev/

**Repositories:**

RTMidi - most popular and lightweight Midi library:

https://github.com/thestk/rtmidi

Mido – python Midi library:

https://mido.readthedocs.io/en/stable/

*(Parser Explanation Articles)*

http://www.petesqbsite.com/sections/express/issue18/midifilespart1.html

http://www.petesqbsite.com/sections/express/issue19/midifilespart2.html

http://www.petesqbsite.com/sections/express/issue20/midifilespart3.html

**Videos:**

Javidx9 – Midi Programming Video:
https://youtu.be/040BKtnDdg0?si=UzKp9w7zF8tkLn6N

Simon Hutchinson – Midi video series:
https://www.youtube.com/@SimonHutchinson